Tarea 2: Subsecuencia de suma máxima

Universidad Autónoma de Querétaro
Instructor: Dr. Arturo González Gutiérrez
Alumno: Ramírez Trejo César Uriel
Matricula: 338042

5 de marzo de 2026

Sección 1

Introducción

Existen algoritmos que no pueden ser ejecutados de manera eficiente con la capacidad de ordenadores de computo personal, es decir que no pueden ser ejecutados en un tiempo razonable, por lo que es necesario contar con herramientas apropiadas para estos casos. El *supercomputo* es la solución a muchos problemas de este tipo permitiendo paralelizar en muchas ocasiones el procesamiento o aumentando la capacidad de computo.

Sección 2

Marco teórico

2.1 Algoritmos

La palabra algoritmo proviene del nombre del matemático persa Al-Khwarizmi, quien escribió un libro sobre la resolución de problemas matemáticos utilizando procedimientos sistemáticos. Un algoritmo es un conjunto de instrucciones o reglas que se siguen para resolver un problema específico o realizar una tarea determinada. En el contexto de la informática, los algoritmos son fundamentales para el desarrollo de software y la resolución de problemas computacionales.

2.2 Big $\mathcal{O}$ notation

Para clasificar la eficacia de un algoritmo medimos como incrementa el gasto de recursos (tiempo y espacio) a medida que el tamaño de la entrada aumenta. A menudo, se suele prestar más atención a la complejidad temporal de los algoritmos sobre la complejidad espacial, debido a que los costos de dispositivos de almacenamiento han disminuido significativamente en comparación con los costos de procesamiento.

Existe para ello un concepto matemático muy útil que nos permite comparar el crecimiento de funciones, la *notación asintótica*, la cual se va a encargar de agrupar nuestras funciones dentro de clases de funciones particulares que nos harán más sencillo el cálculo de la complejidad temporal de un algoritmo

Decimos que una función g es de orden f, denotado $g \in \mathcal{O}(f)$ si

$$g \in \mathcal{O}(f) \Leftrightarrow \exists k > 0 : \exists x_0 \in \mathbb{R} : \forall x \geq x_0 : g(x) \leq kf(x).$$

En la teoría, se le denomina función de complejidad temporal a la función $f : \mathbb{N} \to \mathbb{N}$ que asigna a cada tamaño de entrada n el número máximo de pasos elementales que el algoritmo ejecuta para cualquier entrada de tamaño n.

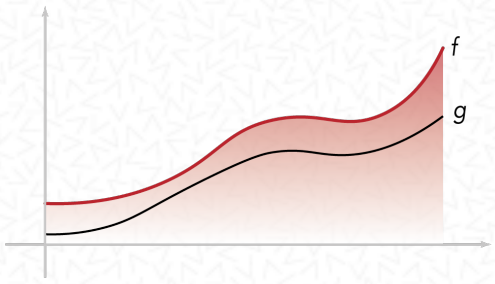


Figura 1: Las funciones g de $\mathcal{O}(f)$ son todas aquellas para las que existe una constante k la gráfica de kf está por encima de la gráfica de g a partir de un cierto punto. Creación propia

La importancia de esta notación radica en que nos permite comparar el crecimiento de funciones, lo que es fundamental para analizar la eficiencia de algoritmos.

2.3 Reducción de clases

Una vez dicha la definición anterior podría pensarse que cada función es la única que vive en su clase de complejidad, sin embargo como se vera en esta subsección existen clases de complejidad importantes que van a simplificar enormemente la tarea del análisis de algoritmos. Para tal fin se enuncian y se demuestran las siguientes proposiciones.

Proposición 1. Todo polinomio de grado α es de orden n^α
Demostración. Sea $p \in \mathbb{P}_\alpha$ entonces

$$\begin{aligned}
p(n) &= \sum_{i=0}^{\alpha} a_i n^i && \text{(Definición)} \\
&\leq \left| \sum_{i=0}^{\alpha} a_i n^i \right| && \text{(No negatividad)} \\
&\leq \sum_{i=0}^{\alpha} \left| a_i n^i \right| && \text{(Desigualdad triangular)} \\
&= \sum_{i=0}^{\alpha} |a_i| \left| n^i \right| && \text{(Propiedad de v.a.)} \\
&\leq \sum_{i=0}^{\alpha} |a_i| \, |n^\alpha| && \text{(Funciones potencia)} \\
&= \left(\sum_{i=0}^{\alpha} |a_i| \right) |n^\alpha| && \text{(Homogeneidad)} \\
&= \left(\sum_{i=0}^{\alpha} |a_i| \right) n^\alpha
\end{aligned}$$

La proposición anterior nos dice que todas las funciones polinomiales pueden reducirse en las clases de complejidad de las funciones potencia.

Proposición 2. Si $b > 1$ y $f : \mathbb{N} \to \mathbb{N}$ entonces $\log_b f(n) \in \mathcal{O}(f(n))$.
Demostración. Notemos que para todo $n \in \mathbb{N}$ se cumple que $n \leq 2^n$ y dado que $\log_2$ es una función creciente, $\log_2 n \leq \log_2 2^n = n \log_2 2 = n$, en consecuencia

$$\frac{\log_2 n}{n} \leq 1 \tag{1}$$

La proposición anterior es válida para $f(n)$ dado que la imagen de f es siempre un número natural. Por otro lado

$$\begin{aligned}
\log_b f(n) &= \frac{\log_2 f(n)}{\log_2 b} && \text{(Cambio de base)} \\
&= \frac{\log_2 f(n)}{\log_2 b} \cdot \frac{f(n)}{f(n)} && \text{(Multiplicación por 1)} \\
&= \frac{\log_2 f(n)}{f(n)} \cdot \frac{f(n)}{\log_2 b} && \text{(Asociatividad)} \\
&\leq \frac{f(n)}{\log_2 b} && \text{(Ecuación 1)} \\
&= k f(n) \quad \square
\end{aligned}$$

Como caso particular tomando $f(n) = n$ tenemos que $\log_b \in \mathcal{O}(n)$ dado que $\log_b n \leq n$, además $n^\alpha \log_b n \leq n^\alpha n = n^{\alpha+1}$ por lo cual $n^\alpha \log_b n \in \mathcal{O}(n^{\alpha+1})$. Más aún, usando la proposición de forma recursiva se puede afirmar que $\log_b(\log_b n) \in \mathcal{O}(\log_b n)$.

Además, se puede probar que:

- Toda función constante está en $\mathcal{O}(1)$.
- Toda formula algebraica simple está en $\mathcal{O}(1)$.
- Toda función acotada está en $\mathcal{O}(1)$ dado que

$$f(n) \leq |f(n)| \leq \lambda := g(n) = 1 \cdot g(n).$$

Proposición 3. Toda función logarítmica (independientemente de su base) es del mismo orden.
Demostración. Sean $a, b \in (1, \infty)$. Entonces, para todo $n \in \mathbb{N}$,

$$\begin{aligned}
\log_a n &= \frac{\log_b n}{\log_b a} && \text{(Cambio de base)} \\
&= \frac{1}{\log_b a} \cdot \log_b n \\
&\leq k \log_b n
\end{aligned}$$

Por lo tanto, $\log_a n \in \mathcal{O}(\log_b n)$ y, de manera análoga, $\log_b n \in \mathcal{O}(\log_a n)$. $\square$

Por simplicidad, se suele escribir $\log n$ para referirse a cualquier función logarítmica, dado que todas son del mismo orden.

Proposición 4. Toda función exponencial es (en sentido estricto) de orden factorial.

Demostración.

$$
\begin{aligned}
n! &= \prod_{i=1}^{n} i && \text{(Definición)} \\
&= \left(\prod_{i=1}^{a} i\right)\left(\prod_{i=a+1}^{n} i\right) && \text{(Descomposición)} \\
&\geq \left(\prod_{i=1}^{a} i\right)\left(\prod_{i=a+1}^{n} a+1\right) && (i \geq a+1) \\
&= \left(\prod_{i=1}^{a} i\right)(a+1)^{n-a} && \text{(Producto constante)} \\
&= \left(\prod_{i=1}^{a} i\right)\frac{(a+1)^n}{(a+1)^a} \\
&= \left[\left(\prod_{i=1}^{a} i\right)\frac{1}{(a+1)^a}\right](a+1)^n \\
&\geq \left[\left(\prod_{i=1}^{a} i\right)\frac{1}{(a+1)^a}\right]a^n \\
&= ka^n.
\end{aligned}
$$

Por lo tanto $a^n \leq \frac{n!}{k}$, es decir, $a^n \in \mathcal{O}(n!)$. □

Podemos sintetizar los ordenes de complejidad estudiados en las proposiciones anteriores de forma ascendente como se muestra en la tabla siguiente. El objetivo en el desarrollo de algoritmos en encontrar uno con el mejor orden de complejidad posible, lo cual ahorra recursos y tiempo de ejecución.

Función	Orden de complejidad
1	Constante
$\log\log n$	Logarítmico doble
$\log n$	Logarítmico
n	Lineal
$n\log n$	Logarítmico lineal
n^a	Polinomial
2^n	Exponencial
$n!$	Factorial

Así como existe esta notación, también existe la notación Ω que se encarga de agrupar funciones que crecen al menos tan rápido como una función dada, y la notación Θ que se encarga de agrupar funciones que crecen exactamente al mismo ritmo que una función dada. Sin embargo, el objetivo central en optimización combinatoria es acotar superiormente las funciones de complejidad temporal superiormente.

2.4 Subsecuencia de suma máxima

Dada una secuencia finita de números cualesquiera, se define una subsecuencia como cualquier secuencia que se pueda obtener de la secuencia dada eliminando algunos o ningún elemento de los extremos de la cadena. El problema de la subsecuencia de suma máxima consiste en encontrar la subsecuencia de una secuencia dada que tenga la suma de sus elementos más grande posible.

Se debe notar que este es un problema de optimización combinatoria. Para verlo, observese que el número de subcadenas de tamaño n es 1, el número de subcadenas de tamaño $n-1$ es 2, el número de subcadenas de tamaño $n-2$ es 3, y así sucesivamente, por lo que el número total de subcadenas es:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Por lo cual, probar exhaustivamente todas las subcadenas posibles es un problema de orden cuadrático, lo cual es insuficiente para cadenas de gran tamaño, aún dejando de lado que no está contemplada la extracción de las subcadenas en este cálculo.

2.5 Complejidad de un algoritmo

Al evaluar la complejidad temporal de un algoritmo se minusprecian muchas operaciones con un coste insignificante en comparación con otras. A pesar de eso al evaluar esto existen algunas reglas interesantes para ello:

- La complejidad de procesos secuenciales es la suma de las complejidades de cada proceso.
- La complejidad de un bucle es la complejidad del proceso dentro del ciclo multiplicado por el número de iteraciones del ciclo.
- La complejidad de procesos condicionales es la complejidad del proceso más costoso.
- La complejidad de procesos recursivos es la complejidad del proceso recursivo multiplicado por el número de llamadas recursivas.

2.6 Tiempos de ejecución

Cuando se desea conocer el tiempo de ejecución de un algoritmo hay un hecho fundamental: *es proporcional a la velocidad promedio de las operaciones*, es decir, si se denota por $t(n)$ el tiempo de ejecución de un algoritmo para resolver un problema de tamaño n, entonces $t(n) \propto f(n)$ y por tanto $f(n) \propto t(n)$ es decir

$$f(n) = \lambda t(n) \tag{2}$$

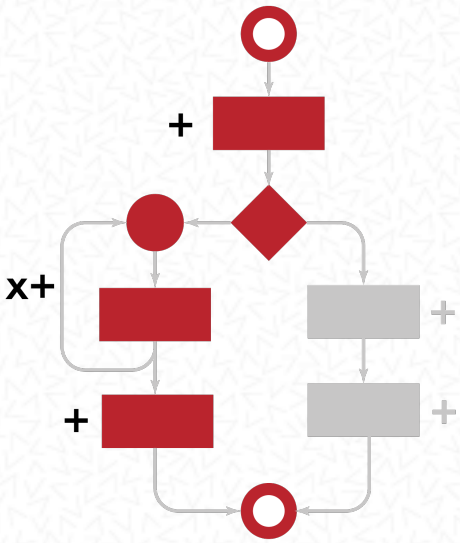


Figura 2: En esta imagen se puede observar el diagrama de flujo de un algoritmo, así mismo en color negro podemos observar como cada componente afecta a la complejidad del mismo, por último de lado derecho podemos observar una rama condicional de menor complejidad "ignorada". Creación propia

o equivalentemente

$$\lambda = \frac{f(n)}{t(n)} \tag{3}$$

Con ayuda de las ecuaciones anteriores es posible establecer una metodología que permita calcular el tiempo de ejecución de un algoritmo sin ejecutarlo.

- **Estimar** λ: Ejecutar el algoritmo con diferentes tamaños de entrada y medir el tiempo de ejecución con algoritmos de complejidad "baja" para estimar el valor λ usando la ecuación 3.
- **Estimar** $t(n)$: Con el valor de λ calculado, es posible usar la ecuación 2 para estimar el tiempo de ejecución de algoritmos con complejidad "alta". Para ello evaluamos el tamaño de la entrada en la función de complejidad del algoritmo y dividimos el resultado por la cantidad promedio de operaciones por segundo λ.

Sección 3

Desarrollo

3.1 Maxsum1

Este algoritmo se enfoca en explorar exhaustivamente todas las subsecuencias existentes y almacenar en una variable en todo momento la subsecuencia que ofrezca el mejor resultado. Para lograrlo, se implementan 3 ciclos anidados que modifican:

- La longitud de la subcadena.
- La posición inicial de la subcadena.
- La suma de los elementos.

Debido a esto se puede calcular que el orden de complejidad del algoritmo es $\mathcal{O}(n^3)$.

3.2 Maxsum2

Este algoritmo es una mejora del algoritmo anterior, dado que el ciclo que se encargaba de calcular la suma de los elementos de la subcadena fue eliminado, y en su lugar se va actualizando la suma a medida que se va modificando la posición inicial y la longitud de la subcadena, lo cual cambia un ciclo por el uso de una simple suma y reduce drasticamente el orden de complejidad a $\mathcal{O}(n^2)$.

3.3 Maxsum3

Este algoritmo sigue la técnica, divide y vencerás, la cual se basa en dividir el problema en subproblemas más pequeños, resolver cada subproblema de manera recursiva y luego combinar las soluciones para obtener la solución del problema original. En este caso, el algoritmo divide la secuencia dada en dos mitades, encuentra la subsecuencia de suma máxima en cada mitad de manera recursiva y luego encuentra la subsecuencia de suma máxima que cruza el punto de división. Finalmente, compara las tres subsecuencias encontradas (la de la primera mitad, la de la segunda mitad y la que cruza el punto de división) y devuelve la que tenga la suma más grande. El orden de complejidad de este algoritmo es $\mathcal{O}(n \log n)$.

Mientras los algoritmos anteriores se enfocaban en explorar todas las subsecuencias posibles, este algoritmo se enfoca en reducir el número de subsecuencias a explorar, lo cual reduce el orden de complejidad.

3.4 Maxsum4

Este algoritmo reduce drasticamente las operaciones a realizar, dado que se enfoca en mantener una suma acumulada de los elementos de la secuencia, y en cada iteración se va actualizando esta suma acumulada, si la suma acumulada no es positiva se reinicia a cero, y se mantiene una variable que almacena la suma máxima encontrada hasta el momento.

Sección 4

Resultados

En esta sección se desarrollarán los experimentos realizados para comparar el desempeño de los algoritmos, así mismo se realizará la estimación de λ y se usarán los resultados obtenidos para estimar el tiempo de ejecución de cada algoritmo para diferentes tamaños de entrada.

4.1 Especificaciones de equipos

Los programas fueron ejecutados simultaneamente en una computadora personal y en un cluster de una supercomputadora. Las especificaciones del equipo de computo personal son las siguientes:

- Procesador: Intel Core i5-13420H (2.1 GHz)
- Memoria RAM: 40 GB de RAM
- Sistema operativo: Windows 11 Home
- IDE: Visual Studio Code
- Lenguaje de programación: C++

Por otro lado, el cluster de la supercomputadora cuenta con las siguientes especificaciones:

- 6 nodos de computo con 186 procesadores en todos los nodos en conjunto.
- Procesador: Intel(R) Xeon(R) E3-1230 (3.50GHz)
- Memoria RAM (Nodo): 16 GB de RAM
- Sistema operativo: Rocky Linux
- Lenguaje de programación: C++

4.2 Experimento 1

Se implementaron en lenguaje C++ 4 algoritmos mencionados previamente, posteriormente se ejecutaron con diferentes tamaños de entrada y se midió el tiempo de ejecución de cada uno en microsegundos (μs). Luego de ello, se graficaron los resultados obtenidos para cada algoritmo en escala logarítmica, lo cual se muestra en la figura 3. Se puede apreciar que el algoritmo Maxsum4 es el que mejor desempeño tiene, seguido por Maxsum3, luego Maxsum2 y por último Maxsum1, tal y como se esperaba dado el orden de complejidad de cada algoritmo.

Con una línea punteada se muestra el tiempo que corresponde a una hora de duración de ejecución ($3.6 \times 10^9 \mu$s). Por otro lado, se puede observar una mejor "estabilidad" en el desempeño de los algoritmos cuando fueron implementados en la supercomputadora lo cual puede deberse al hecho de que la computadora personal utilizada no estaba dedicada exclusivamente a la ejecución de los algoritmos, razón por la cual se pueden observar algunos picos en el tiempo de ejecución de los algoritmos.

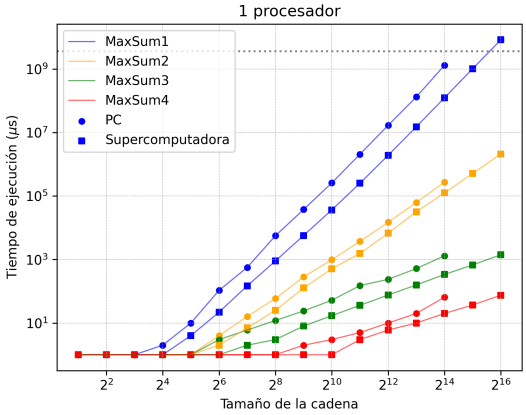


Figura 3: Se puede apreciar el desempeño de los algoritmos tanto en una computadora de uso personal como un cluster de una supercomputadora (sin aplicar ningún tipo de paralelización). Gráfico generado con Python

4.3 Estimación de λ

Para estimar el valor de λ se ejecutaron los algoritmos con diferentes tamaños de entrada y se midió el tiempo de ejecución, concretamente se obtuvo con los datos obtenidos de la ejecución del algoritmo anterior, posteriormente se calculó el valor de λ usando la ecuación 3 y se obtuvo un valor de

$$\lambda \approx 1.31 \cdot 10^9$$

operaciones por segundo en promedio, lo cual es congruente comparado con los 2.1 GHz de velocidad anunciados por el fabricante del procesador utilizado para la ejecución de los algoritmos.

Se calculó el valor de λ usando la ecuación 3 para cada instancia obtenida de la ejecución de los algoritmos anteriores y se graficaron los resultados como un gráfico de dispersión, lo cual se muestra en la figura 4, también ahí aparece una línea punteada que representa el valor promedio de λ obtenido a partir de todas las instancias. Este gráfico nos permite visualizar la variabilidad de las estimaciones de λ y cómo se comparan con el valor promedio. ejecución del algoritmo anterior

en cada instancia, los puntos corresponden a cada instancia y su respectivo valor de λ, mientras que la línea punteada corresponde al valor promedio de λ obtenido.

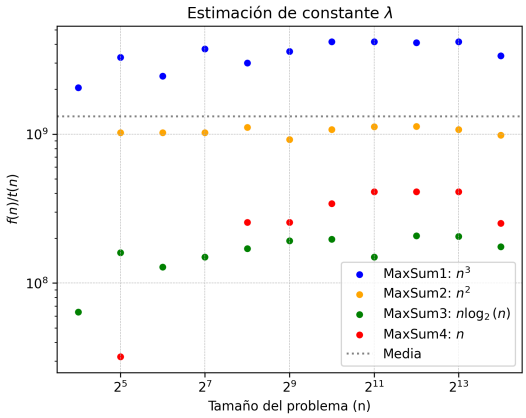


Figura 4: Gráfico generado con Python

4.4 Estimaciones de tiempo de ejecución

Una vez calculado el valor λ, ahora es posible usar la ecuación 2 para estimar el tiempo de ejecución de cada algoritmo para diferentes tamaños de entrada. Para ello se proponen distintos tamaños de entrada y funciones de complejidad, se evaluó el tamaño de la entrada en las funciones y se dividió el resultado por la constante λ, posteriormente se obtuvo el tiempo estimado de ejecución y el mismo fue organizado en la tabla **??**.

4.5 Paralelización por hilos

Para disminuir el tiempo de ejecución de los algoritmos, se implementó una versión paralela de cada algoritmo utilizando hilos, lo cual permite que diferentes partes del algoritmo se ejecuten simultáneamente en diferentes núcleos del procesador. En esta sección vamos a medir los tiempos de ejecución con diferente cantidad de hilos ejecutando el código simultaneamente en la supercomputadora.

Cómo se puede observar cuando se ejecuta un algoritmo de complejidad cúbica el desempeño de la paralelización ofrece una mejora significativa, sin embargo, a medida que el tamaño de la entrada aumenta, la diferencia en el desempeño es cada vez menos notoria. Por otro lado, al ejecutar el algoritmo en un solo procesador, el tiempo aumenta constantemente. Superando para una entrada de tamaño 65,000 el tiempo de ejecución de una hora.

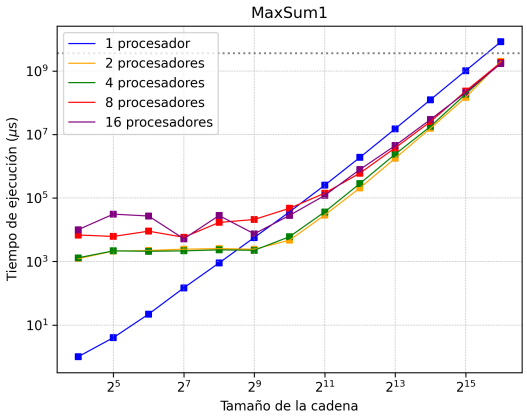


Figura 5: Gráfico generado con Python

Cuando ejecutamos un algoritmo de complejidad cuadrática, el desempeño de la paralelización es significativamente menos eficiente que en el caso anterior, sin embargo, a medida que el tamaño aumenta puede observarse una clase de acotación que muestra que el tiempo de ejecución tiende a homologarse a medida que el tamaño de la entrada aumenta, excepto cuando se utiliza un solo procesador.

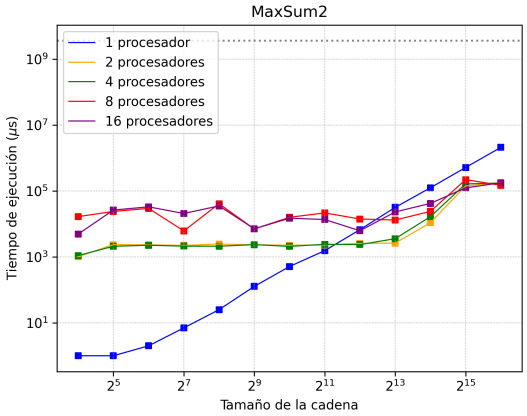


Figura 6: Gráfico generado con Python

En el caso de un algoritmo de complejidad logarítmica lineal, el desempeño de la paralelización es similar en todos los casos, sin embargo, a medida que el tamaño de la entrada aumenta, la diferencia en el desempeño es cada vez menos notoria.

Podemos observar un caso similar ahora con un algoritmo de complejidad lineal, sin embargo, sucede algo revelador, la paralelización es menos eficiente que utilizar un solo procesador, lo cual puede deberse a que el costo de la paralelización es mayor que el beneficio

Tamaño	Lineal	Logarítmico lineal	Cuadrático	Cúbico	Exponencial	Factorial
10	7.69 ns	17.71 ns	76.92 ns	769.23 ns	787.69 ns	2.79 ms
25	19.23 ns	89.28 ns	480.76 ns	12.01 μs	25.79 μs	378,561 años
50	38.46 ns	217.09 ns	1.92 μs	96.15 μs	27.42 años	-
100	76.92 ns	511.06 ns	7.69 μs	769.23 μs	3×10^{13} años	-
200	153.84 ns	1.17 μs	30.76 μs	6.15 ms	3×10^{43} años	-
400	307.69 ns	2.65 μs	123.07 μs	49.23 ms	-	-
800	615.38 ns	6.01 μs	492.30 μs	393.84 ms	-	-
1,000	769.23 ns	7.69 μs	769.23 μs	769.23 ms	-	-
5,000	3.84 μs	47.16 μs	19.23 ms	1.60 min	-	-
10,000	7.69 μs	102.21 μs	76.92 ms	12.82 min	-	-
50,000	38.46 μs	600.32 μs	1.92 s	26.70 h	-	-
100,000	76.92 μs	1.28 ms	7.69 s	8.89 días	-	-
500,000	384.61 μs	7.28 ms	3.20 min	3.04 años	-	-
1,000,000	769.23 μs	15.33 ms	12.82 min	24.38 años	-	-

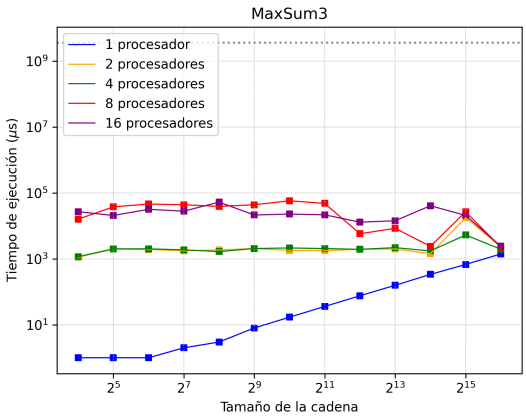


Figura 7: Gráfico generado con Python

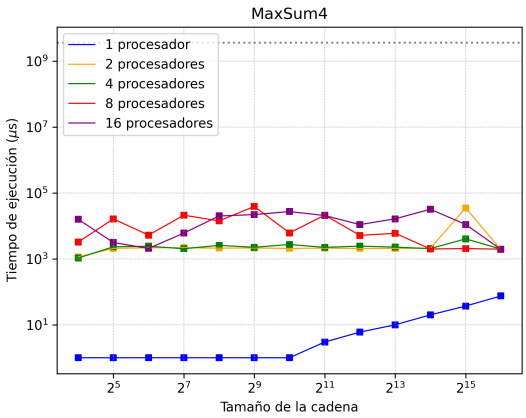


Figura 8: Gráfico generado con Python

que se obtiene al utilizar varios procesadores, lo cual es un caso de estudio interesante a analizar.

4.6 Algoritmo propuesto

En esta sección se desarrolla un algoritmo de complejidad lineal para resolver el problema de la subsecuencia de suma máxima diferente a aquellos propuestos previamente.

Proposición 1. La subcadena de suma máxima de una secuencia dada es la misma que si restamos una constante a cada elemento de la secuencia original la diferencia aplicada multiplicada por la longitud de la subcadena.

Demostración. Podemos pensar matemáticamente en el problema de la subsecuencia de suma máxima como sigue: Dada una "secuencia" $(x_0, x_1, \dots, x_{n-1}) \in \mathbb{R}^n$ el problema consiste en hallar un subconjunto de índices $i, j \in \{0, 1, \dots, n-1\}$ tal que la suma $\sum_{k=i_1}^{i_2} x_k \geq \sum_{k=j_1}^{j_2} x_k$ para todo $(j_1, j_2) \in \{0, 1, \dots, n-1\} \times \{0, 1, \dots, n-1\}$. Dicho

lo anterior, la proposición establece entonces que

$$S_1 := \sum_{k=i_1}^{i_2} x_k \geq \sum_{k=j_1}^{j_2} x_k$$
$$\Rightarrow \sum_{k=i_1}^{i_2} (x_k - c + c) \geq \sum_{k=j_1}^{j_2} (x_k - c + c)$$
$$\Rightarrow \sum_{k=i_1}^{i_2} (x_k - c) + \sum_{k=i_1}^{i_2} c \geq \sum_{k=j_1}^{j_2} (x_k - c + c) + \sum_{k=i_1}^{i_2} c$$
$$\Rightarrow \sum_{k=i_1}^{i_2} (x_k - c) + (i_2 - i_1 + 1)c \geq \sum_{k=j_1}^{j_2} (x_k - c) + (i_2 - i_1 + 1)c$$

La proposición anterior nos dice que podemos encontrar la subcadena de suma máxima de una secuencia dada, trabajando sobre otra cadena, una que nos favorezca y luego transformarla.

Ahora toca el turno de pensar en el problema de forma continua y luego aplicarlo sobre nuestro problema. Para ello, vamos a considerar una función $f : [a, b] \to \mathbb{R}$ continua y vamos a encontrar la región de área máxima, es decir el problema consiste en encontrar $c, d \in [a, b]$ tales que $c < d$ y que verifiquen que

$$\int_c^d f(x)\,dx \geq \int_\alpha^\beta f(x)\,dx$$

para cualesquiera $\alpha, \beta \in [a, b]$. De esta manera el problema se ha transformado en maximizar la función $g : [a, b] \times [a, b] - \{(c, d) \in \mathbb{R}^2 : d < c\} \to \mathbb{R}$ dada por

$$g(\alpha, \beta) = \int_\alpha^\beta f(x)\,dx$$

la cual representa un campo escalar de dos variables, por lo cual puede ser atacada por medio de técnicas de optimización del cálculo vectorial, comenzando por encontrar los puntos críticos de la función g, es decir puntos en los cuales el gradiente de g es el vector nulo, es decir, $\nabla g(c, d) = \vec{0}$, es decir puntos del dominio donde las derivadas parciales se anulan.

Para lo cual

$$\begin{aligned}\frac{\partial g}{\partial \alpha} &= \frac{\partial}{\partial \alpha}\left(\int_\alpha^\beta f(x)\,dx\right)\\ &= \frac{\partial}{\partial \alpha}\left(F(\beta) - F(\alpha)\right)\\ &= \frac{\partial}{\partial \alpha}(F(\beta)) - \frac{\partial}{\partial \alpha}(F(\alpha))\\ &= -f(\alpha).\end{aligned}$$

Similarmente se puede probar que $\frac{\partial g}{\partial \beta} = f(\beta)$, por lo cual el gradiente es nulo siempre que $\alpha, \beta \in \{x \in [a, b] : f(x) = 0\}$.

Es decir, hemos probado que el valor máximo de g se obtendra dentro de intervalos donde los extremos del intervalo sean ceros de la función f. Y eso es justo lo que necesitamos, dado que por hipótesis los números son enteros generados de forma pseudoaleatoria, en teoría, no habrá tantos ceros y podemos encontrar la subcadena de suma máxima explorando únicamente las subcadenas que comienzan y terminan en ceros, lo cual reduce el número de subcadenas a explorar considerablemente.

Para garantizar que existan ceros podemos aplicar como constante de diferencia el valor entero más próximo a la media de los números generados.

Por tanto el algoritmo queda de la siguiente manera:

1. Se calcula la media del vector x
2. Se le resta a cada elemento del vector x la media calculada, si los valores son flotantes, se calcula el valor más cercano de la media a x y se le resta ese valor, eso garantiza que haya ceros.
3. Se busca la subcadena de suma máxima explorando únicamente las subcadenas que comienzan y terminan en ceros.
4. Se le suma la media calculada a cada elemento de la subcadena encontrada para obtener la subcadena original con la suma máxima.

Dado que el proceso más costoso es verificar sobre cada combinación de ceros distintos tenemos que la comlejidad del algoritmo es $\mathcal{O}(m^2)$ donde m es el número de ceros encontrados. En trabajos futuros puede discutirse si realmente representa una mejora, dado que a pesar de que hablamos de un algoritmo de orden cuadrático, el número de ceros en teoría debe ser pequeño por como se generan los números.

Sección 5

Discusión de resultados

Cuando se desarrollan algoritmos una parte fundamental del estudio además de verificar la funcionalidad es analizar la complejidad algorítimica temporal y espacial, dado que en la mayoría de las ocasiones suele bastarnos con verificar la funcionalidad aunque posiblemente el algoritmo desarrollado no sea el más

eficiente, y como vimos ahora, fácilmente se pueden encontrar algoritmos "simples" con un tiempo de ejecución inmenso.

Si bien es cierto que la paralelización nos ofrece una mejora significativa en el tiempo de ejecución de los algoritmos, también es cierto que no siempre es la mejor opción dado que el tiempo de paralelización (separar las tareas y una vez resueltas las tareas combinar las soluciones) en muchas ocasiones requiere un esfuerzo mayor que lo que tomaría no haberlos paralelizado.

Por último, es importante considerar que la mejora en el desempeño al paralelizar la tarea se ve acotada en todos los casos, en parte es lo que es estudiado por el teorema de Amdahl, el cual establece los límites teóricos de la paraleliazación de algoritmos. Dicho lo anterior, es impresindible analizar cada caso en particular para determinar si la paralelización es una buena opción o no, dado que en algunos casos puede ser contraproducente.

Sección 6

Agradecimientos

Se agradece al Dr. Arturo González Gutiérrez por su apoyo y guía en el desarrollo de esta tarea, así como al Laboratorio Nacional de Visualización Científica Avanzada (LAVIS) de la Universidad Nacional Autónoma de México (UNAM) por proporcionar los recursos computacionales necesarios para la ejecución de los algoritmos desarrollados así como la capacitación para el uso de los mismos.

Referencias

[1] Michael R. Garey y David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman y Company, 1979.

[2] Teofilo F. Gonzalez, ed. *Handbook of Approximation Algorithms and Metaheuristics*. Chapman & Hall/CRC, 2007.